LEARN PYTHON PROGRAMMING – BEGINNER TO MASTER

Section: Sec 1. Introduction To Python

Lecture: 4. Compiler Vs Interpreter

Section 1: Lecture Summary

This lecture explains the fundamental difference between **compilers** and **interpreters**, two main methods of program translation from high-level programming languages to machine-readable code. Using a natural language analogy of translating a Chinese recipe for an English speaker, it clarifies the key characteristics and workflow of compilers and interpreters. The lecture then relates these concepts directly to programming languages like C (compiler-based) and JavaScript (interpreter-based), highlighting how they translate and execute code differently. The comparison covers aspects such as one-time vs repeated translation, creation of separate executable files, simultaneous translation and execution, dependency on a translation tool during execution, and error handling behavior. The lecture concludes by noting that Python is a hybrid language with both compiler and interpreter traits.

Section 2: Key Concepts and Explanations

- **Two Methods of Translation**

  - **Compiler**: Translates the entire source code into machine code (binary) once, creating a separate executable file. After this one-time translation, the program runs independently without needing the compiler again.

  - **Interpreter**: Translates and executes the program line-by-line at runtime, translating simultaneously as the program runs. No separate executable file is created, and the original source code remains visible.

- **Natural Language Analogy**

  - Compiler example: Translate a Chinese recipe fully into English first, write it down, and then follow the English steps independently.

- Interpreter example: A friend reads and translates the Chinese recipe step-by-step orally as the dish is being prepared, requiring continual presence for each preparation.

- **Comparative Differences Between Compiler and Interpreter**

Aspect	Compiler	Interpreter
Translation frequency	One-time translation (before execution)	Repeated translation every time execution occurs
Output	Separate machine code file (executable)	No separate file, source code remains as-is
Execution sequencing	Execution starts after complete translation	Translating and executing happen simultaneously (line-by-line)
Dependency during execution	Independent execution after compilation	Dependent on the interpreter during execution
Error handling	If error, no executable generated, no run	Partial execution until error occurs
Speed	Generally faster execution	Slower, as code is translated on the fly
Learning curve & usage	Typically general-purpose, harder to learn	Easier for scripting, often special-purpose languages

- **Programming Language Examples**

- **C language**: Compiler-based. `.c` source code compiles to an independent machine code executable file, run without compiler present.

- **JavaScript**: Interpreted by the browser at runtime; source code is visible and executed inside the runtime environment line-by-line.

- **Python**

- A hybrid language combining compilation and interpretation traits.

- Often called interpreted, but it produces intermediate bytecode which is then interpreted.

- General-purpose language suited for many types of programming.

Section 3: Example Code and Use Cases

Example illustrating compilation concept (C language analog)

C source code (sample.c):

```
int main() {

printf("Hello, World!");

return 0;

}
```

After compilation, machine code (.exe or binary) is generated and run independently.

Explanation: This C program is compiled once into machine code and executed independently without requiring the source code or compiler during runtime.

```
// Example illustrating interpretation concept (JavaScript)
console.log("Hello, World!");
```

Explanation: JavaScript code runs inside a browser interpreter which reads and executes each line. The source code is visible in the browser (e.g., via Inspect Element), and the translation (interpretation) happens every execution time line-by-line.

Section 4: Key Takeaways

- A **compiler** translates entire source code to machine code once and produces an independent executable file. Execution happens after full translation.

- An **interpreter** translates and executes the program simultaneously line-by-line every time the program runs, without creating an independent executable.

- Compiler-based languages like C create a machine code file that can run without the compiler again; interpreted languages like JavaScript require the interpreter (e.g., browser) every time.

- Interpreted languages typically execute slower due to repeated translation during runtime but are easier for scripting and quick edits.

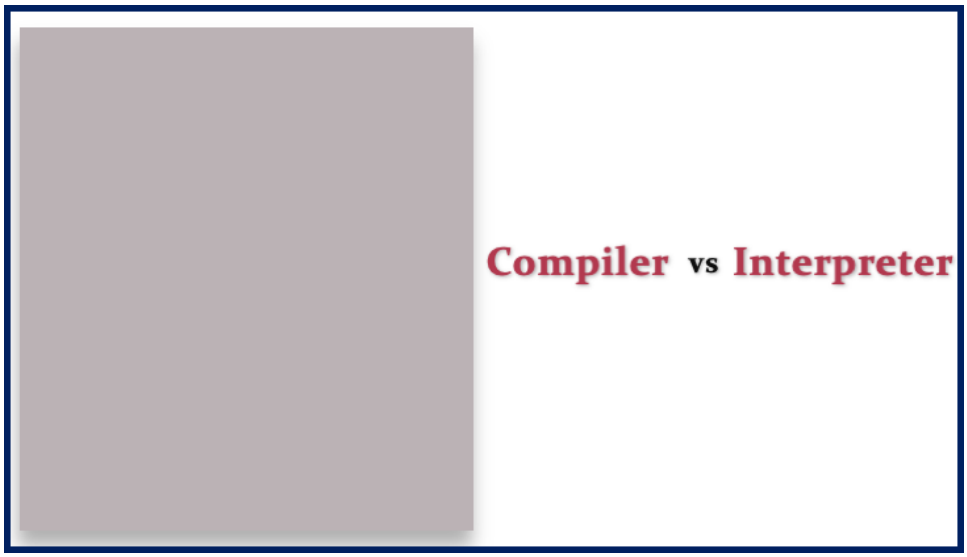
- If errors exist, compilers halt before creating executable; interpreters run code until the error line.

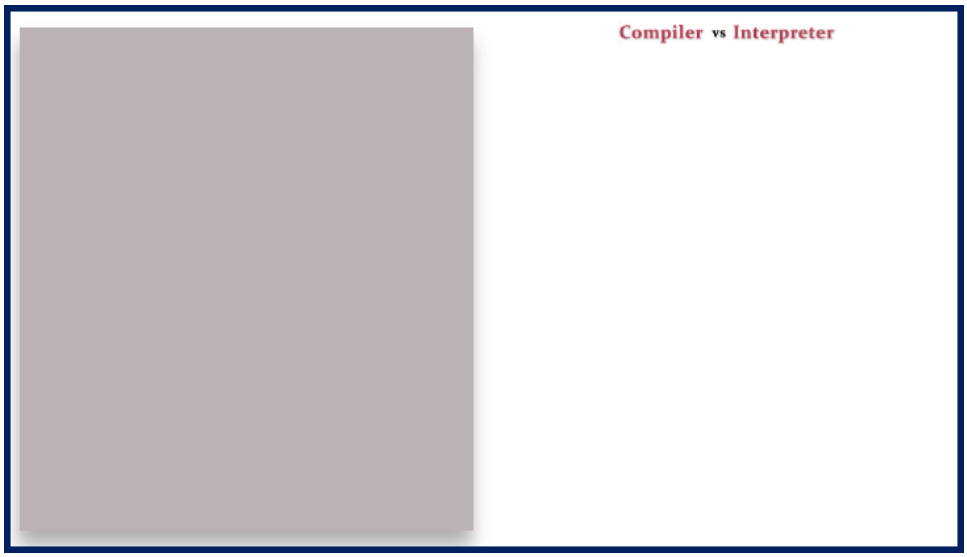
- Python is a hybrid language that mixes both compilation and interpretation processes, giving it flexibility and general-purpose usability.

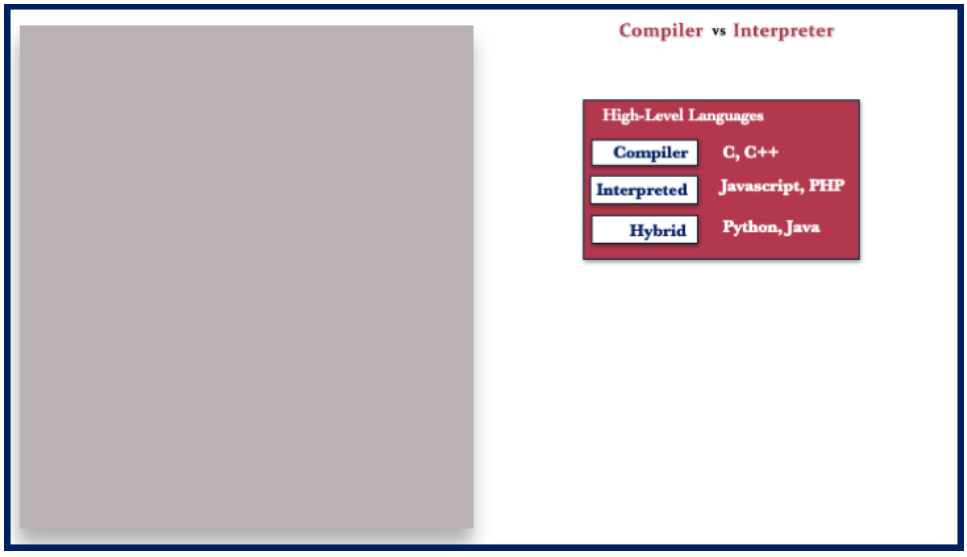
- Understanding these differences helps clarify how different programming languages execute code and manage errors, which is essential for programming proficiency.

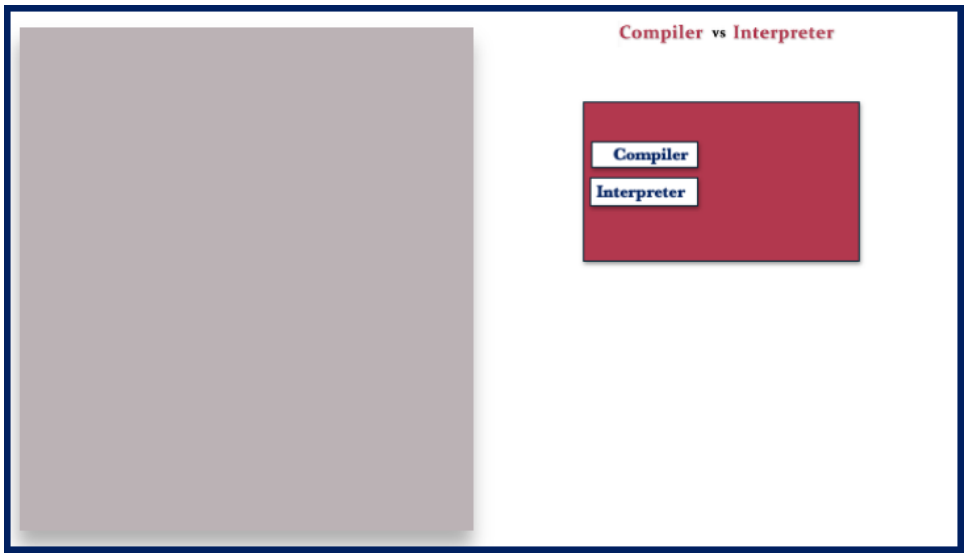
Lecture in Slides

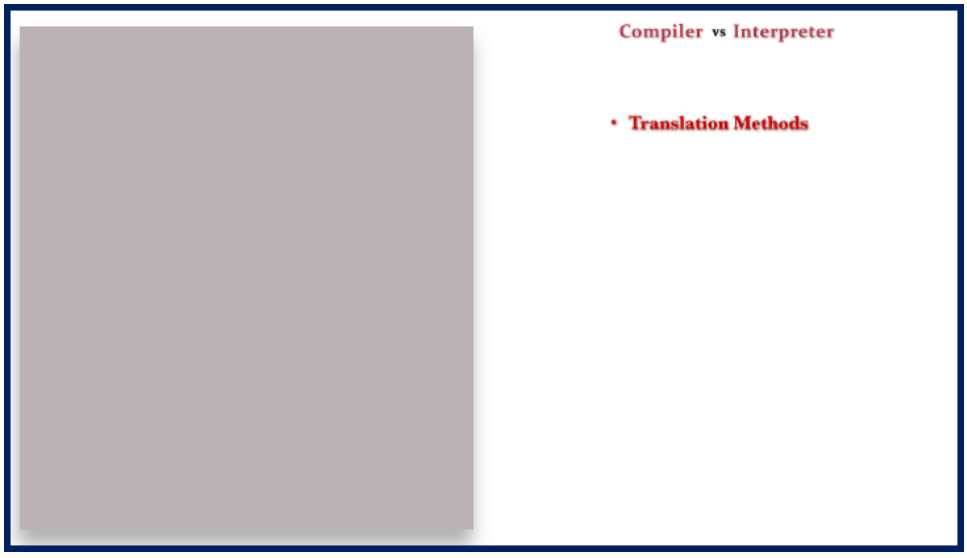
Please Note: This document presents a curated selection of slides from the lecture; others have been excluded for conciseness.

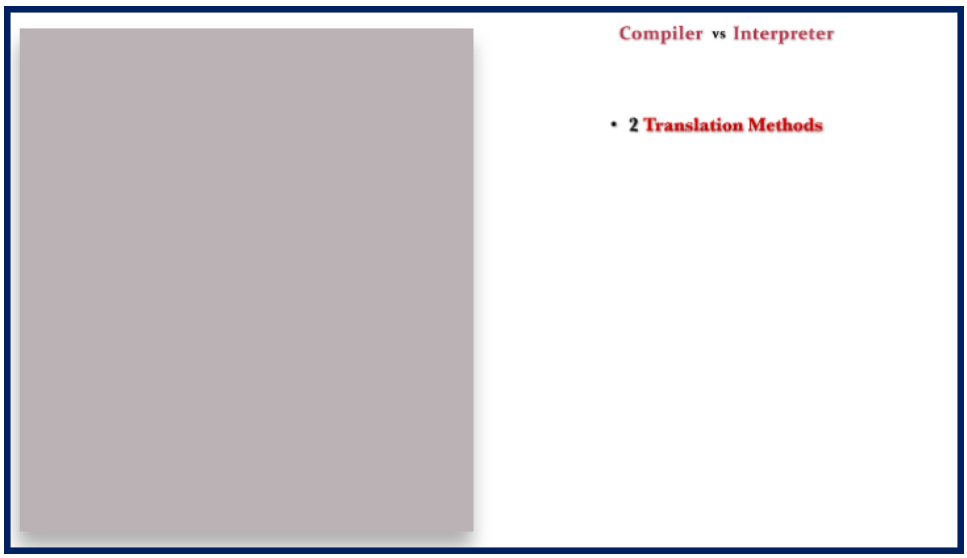


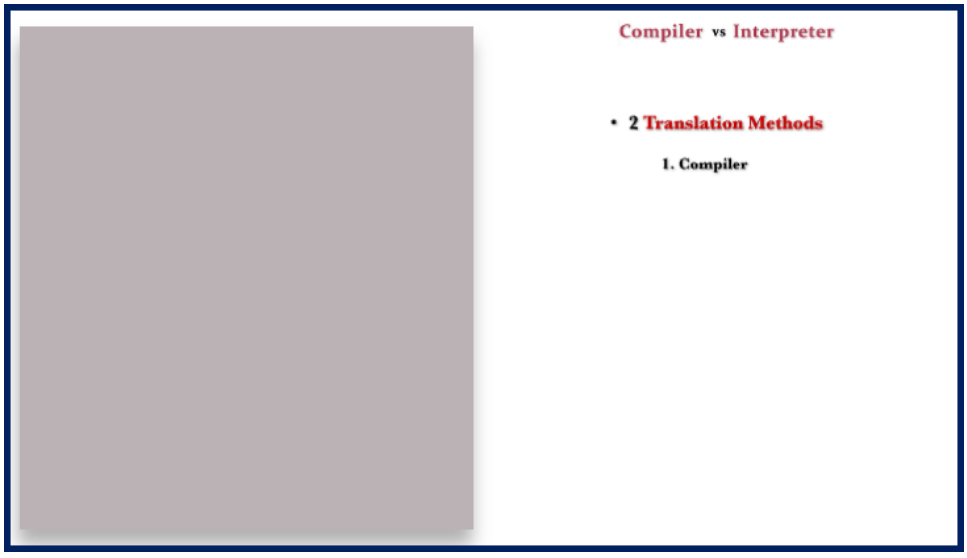

Compiler vs Interpreter

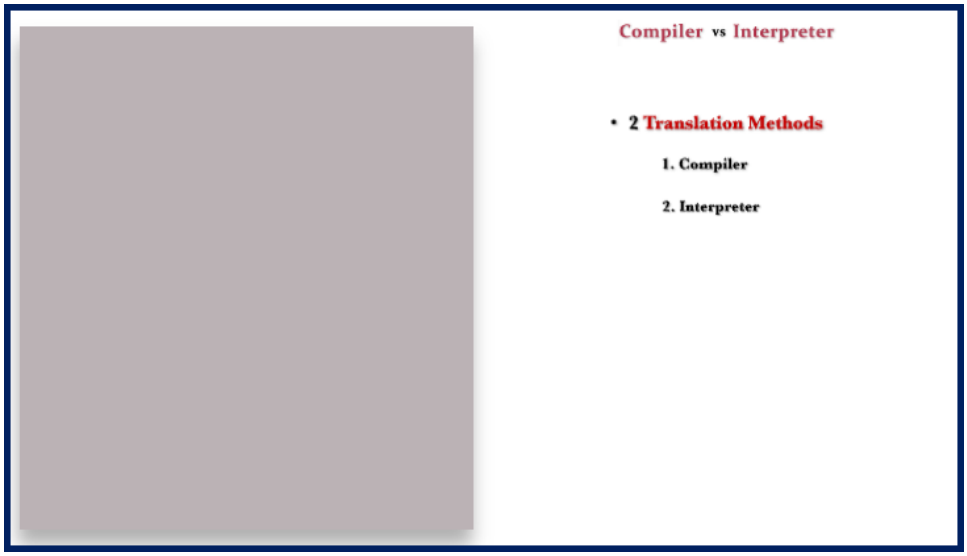

Compiler vs Interpreter
High-Level Languages
Compiler
C, C++
Interpreted
Javascript, PHP
Hybrid
Python, Java

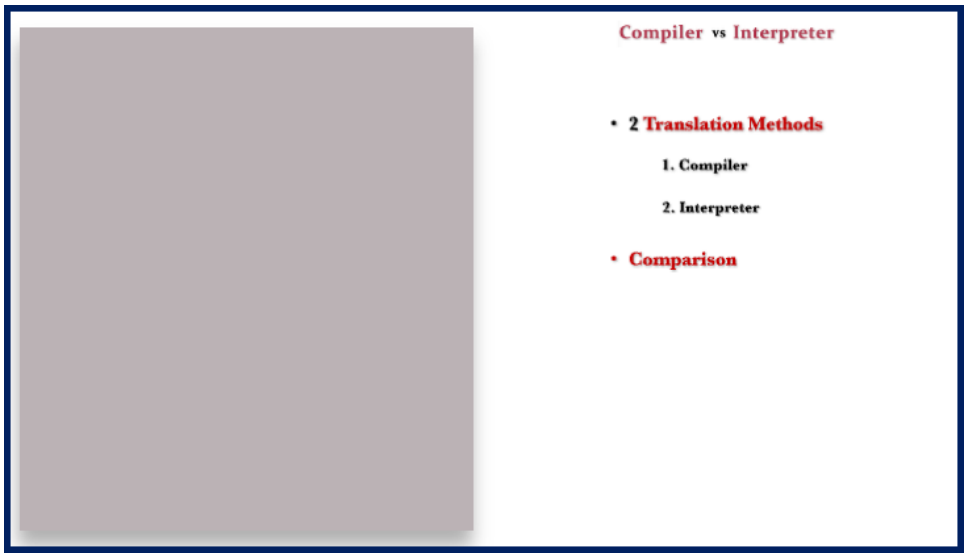

Compiler vs Interpreter
Compiler
Interpreter


Compiler vs Interpreter
• Translation Methods


Compiler vs Interpreter
• 2 Translation Methods


Compiler vs Interpreter
• 2 Translation Methods
1. Compiler


Compiler vs Interpreter
• 2 Translation Methods
1. Compiler
2. Interpreter


Compiler vs Interpreter
• 2 Translation Methods
1. Compiler
2. Interpreter
• Comparison

Compiler vs Interpreter

Compiler vs Interpreter

Chinese

Compiler vs Interpreter

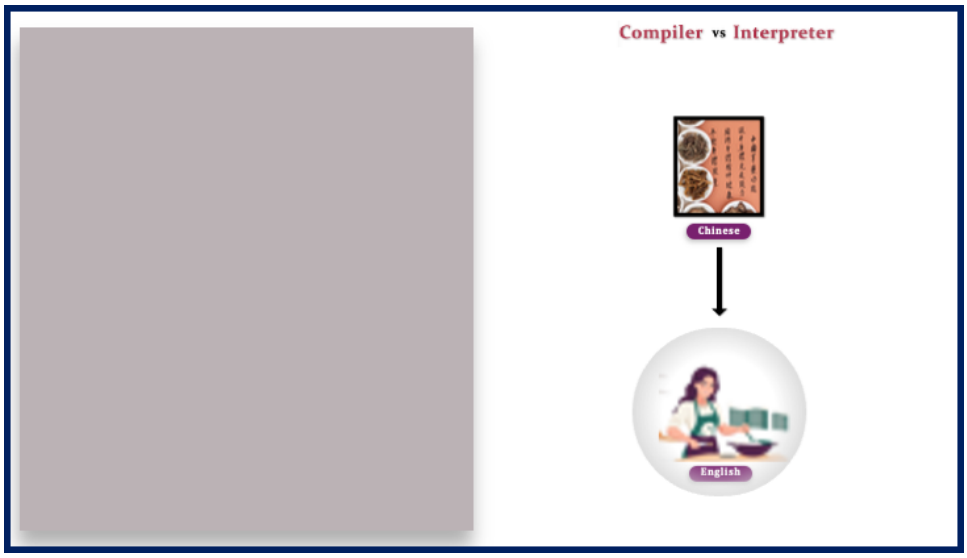
Chinese

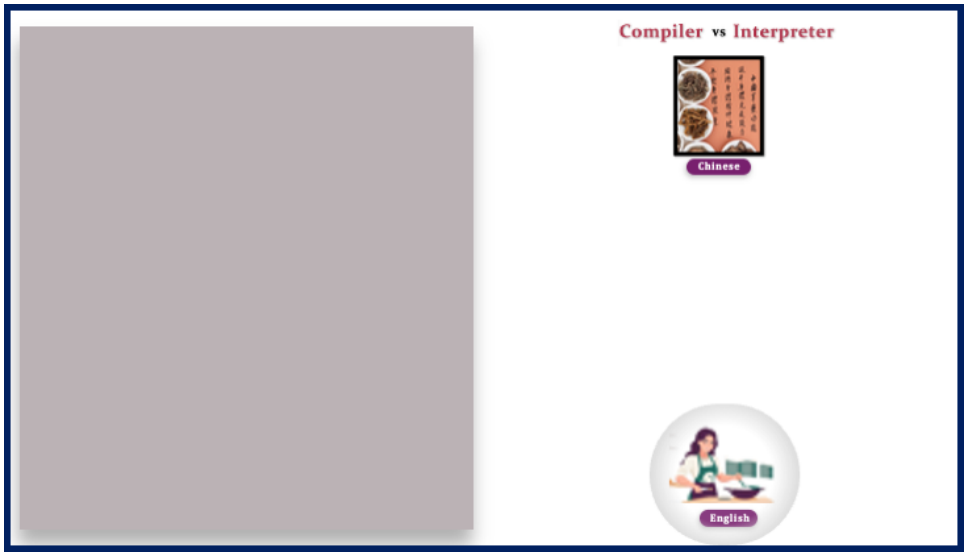
Compiler vs Interpreter

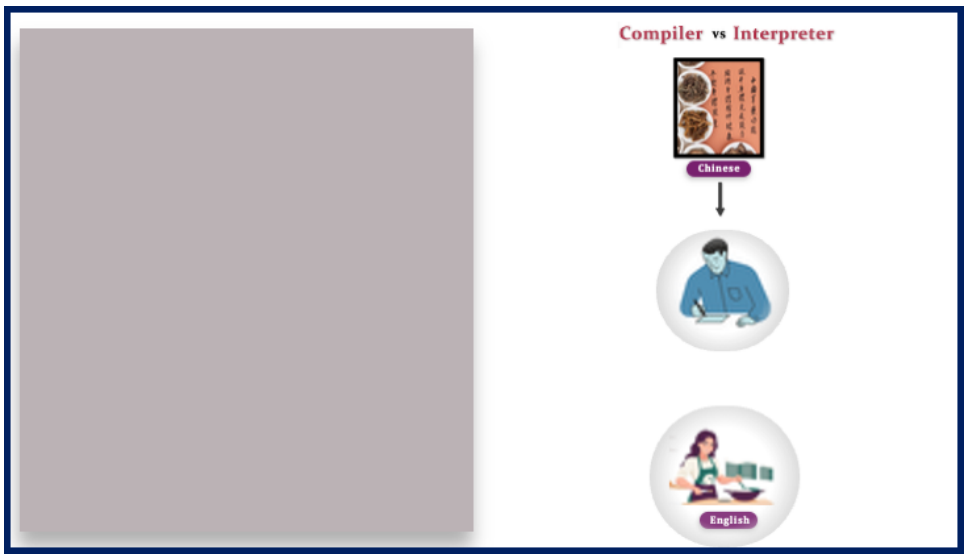
Chinese

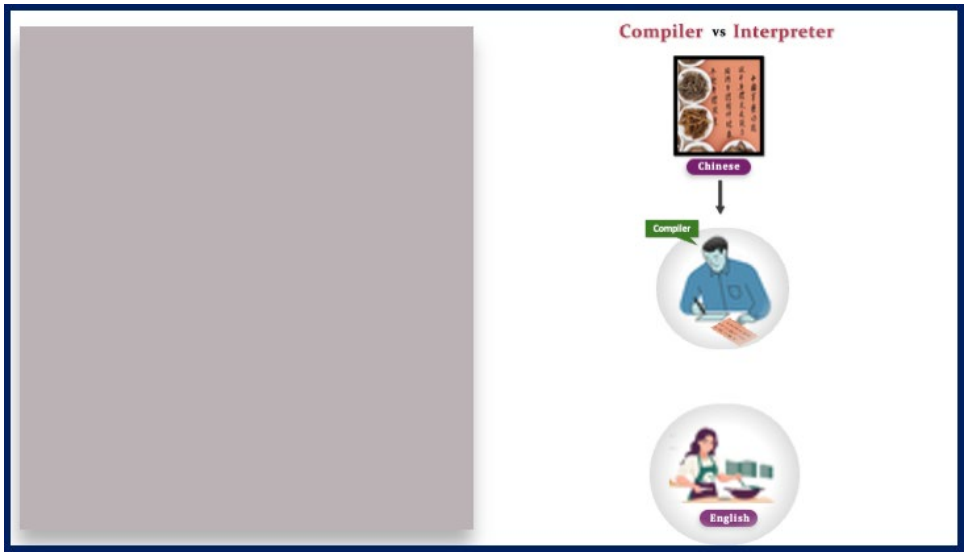
Compiler vs Interpreter

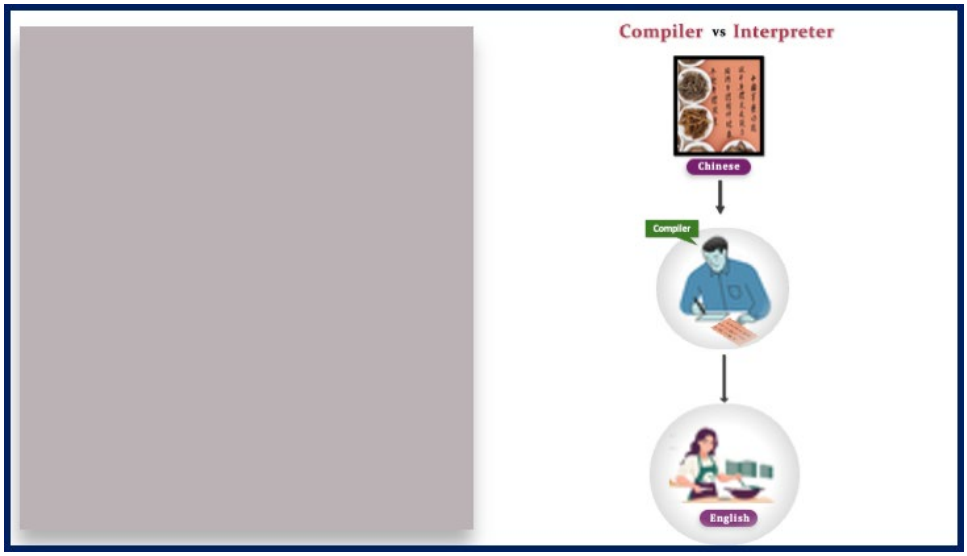
Chinese

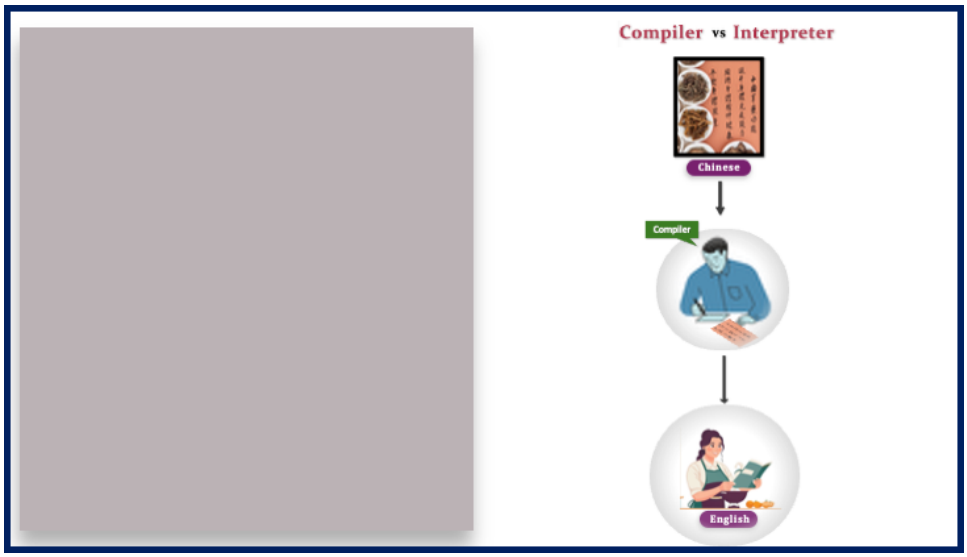

Compiler vs Interpreter
Chinese
English

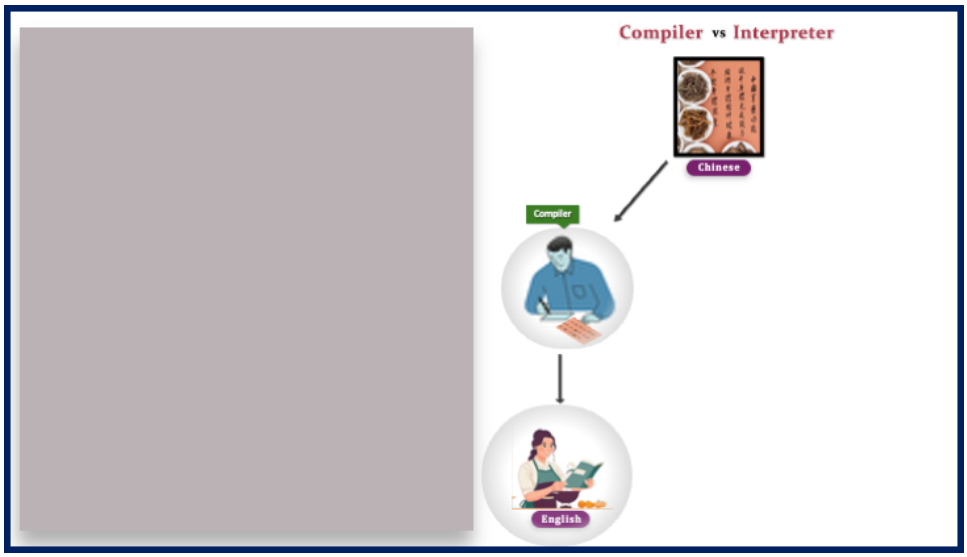

Compiler vs Interpreter
Chinese
English

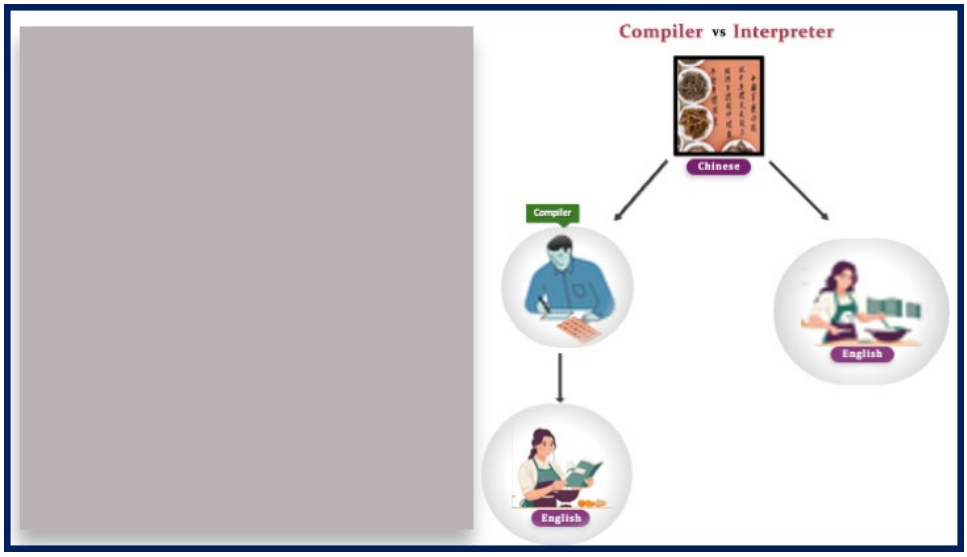

Compiler vs Interpreter
Chinese
English

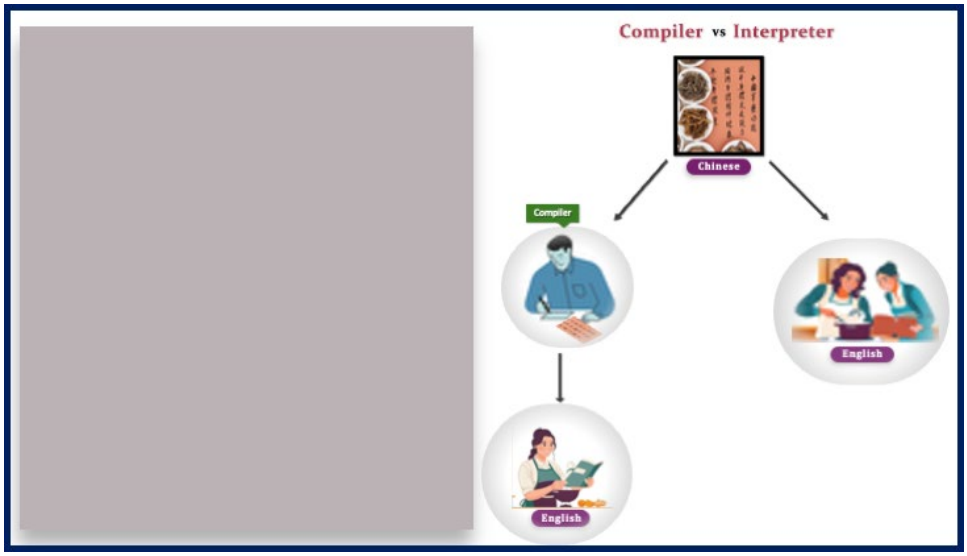

Compiler vs Interpreter
Chinese
Compiler
English

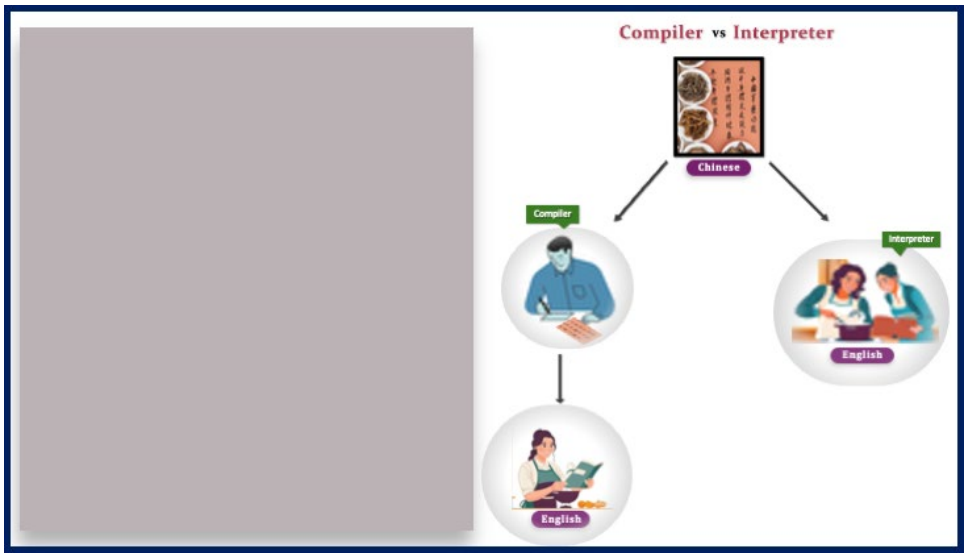

Compiler vs Interpreter
Chinese
Compiler
English

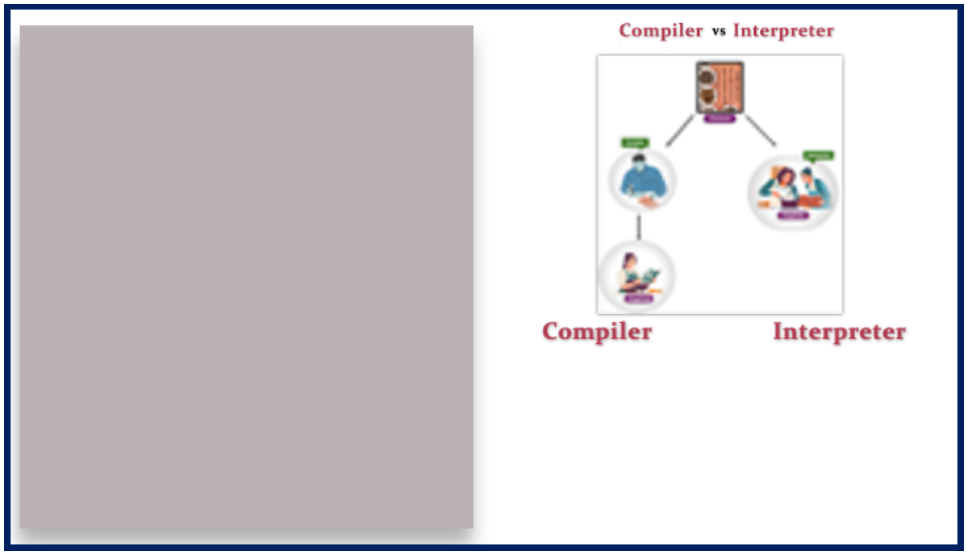

Compiler vs Interpreter
Chinese
Compiler
English


Compiler vs Interpreter
Chinese
Compiler
English


Compiler vs Interpreter
Chinese
Compiler
English
English


Compiler vs Interpreter
Chinese
Compiler
English
English


Compiler vs Interpreter
Chinese
Compiler
Interpreter
English
English


Compiler vs Interpreter
Compiler
Interpreter

Compiler vs Interpreter

Compiler

- One time Translation

Interpreter

Compiler vs Interpreter

Compiler

- One time Translation

Interpreter

- 'N' time Translation

Compiler vs Interpreter

Compiler

- One time Translation
- Copy of Recipe in English

Interpreter

- 'N' time Translation

Compiler vs Interpreter

Compiler

- One time Translation
- Copy of Recipe in English

Interpreter

- 'N' time Translation
- Recipe still in Chinese

Compiler vs Interpreter

Compiler

- One time Translation
- Copy of Recipe in English
- Preparation after Translation

Interpreter

- 'N' time Translation
- Recipe still in Chinese

Compiler vs Interpreter

Compiler

- One time Translation
- Copy of Recipe in English
- Preparation after Translation

Interpreter

- 'N' time Translation
- Recipe still in Chinese

- Preparation while Translation

Compiler vs Interpreter

Compiler	Interpreter
• One time Translation	• 'N' time Translation
• Copy of Recipe in English	• Recipe still in Chinese
• Preparation after Translation	• Preparation while Translation
• Independent Preparation	

Compiler vs Interpreter

Compiler	Interpreter
• One time Translation	• 'N' time Translation
• Copy of Recipe in English	• Recipe still in Chinese
• Preparation after Translation	• Preparation while Translation
• Independent Preparation	• Preparation with Help

Compiler vs Interpreter

Compiler

- One time Translation
- Copy of Recipe in English
- Preparation after Translation
- Independent Preparation
- If Errors, no Translation

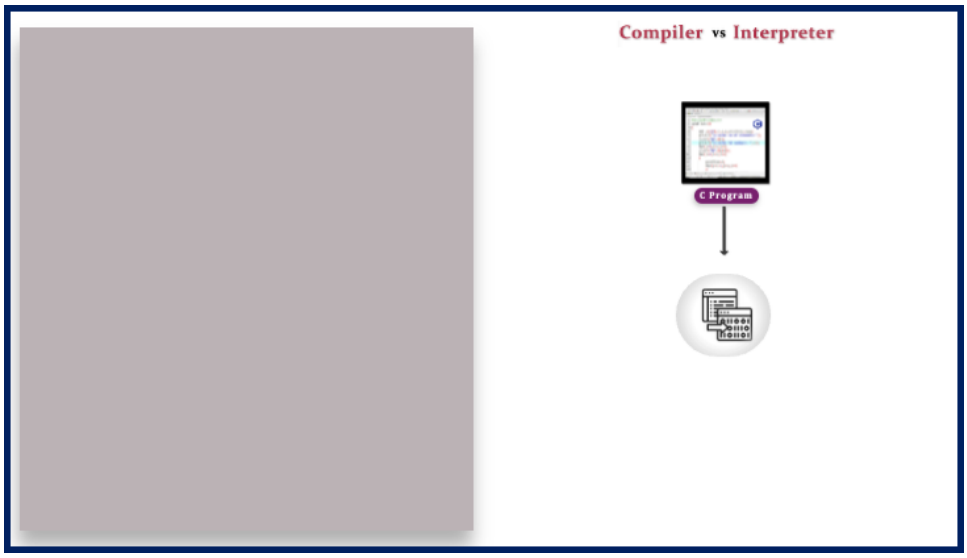
Interpreter

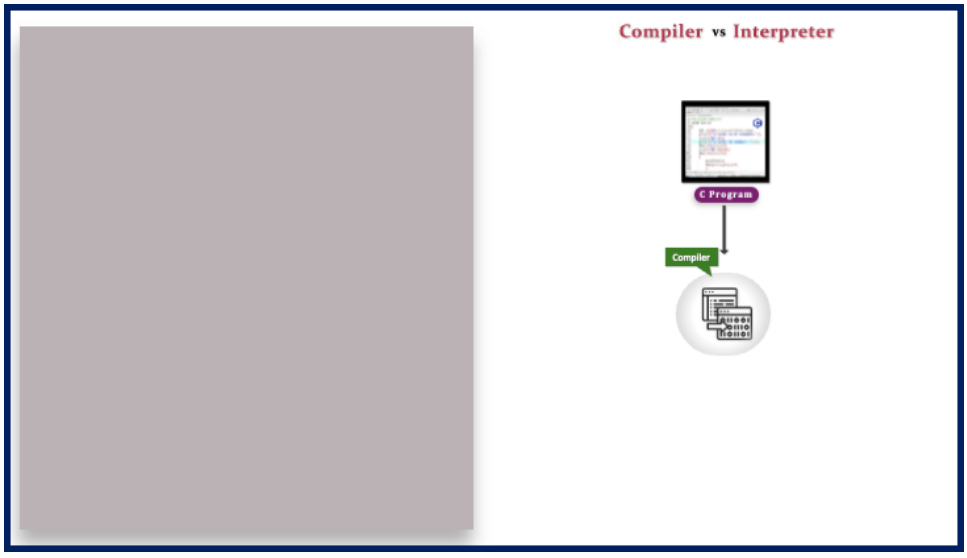
- 'N' time Translation
- Recipe still in Chinese
- Preparation while Translation
- Preparation with Help

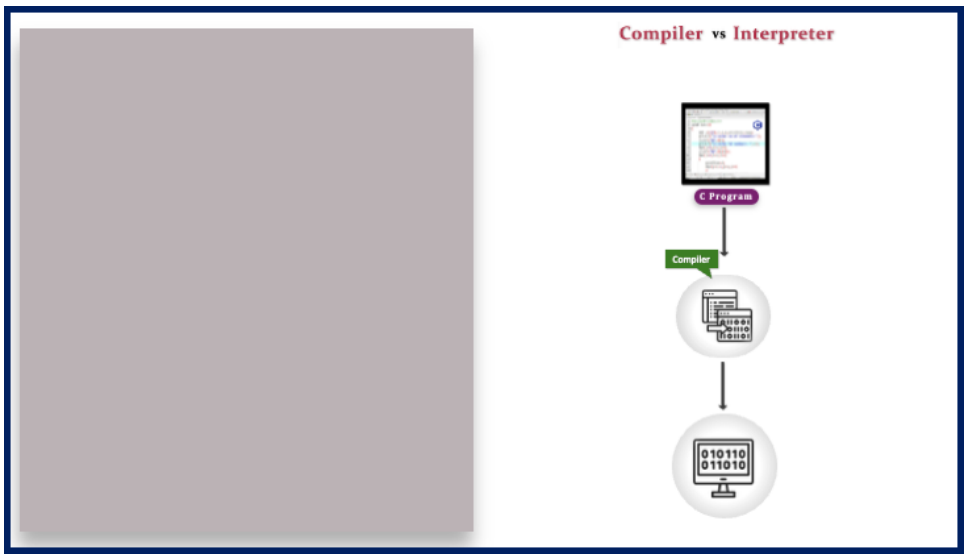
Compiler vs Interpreter

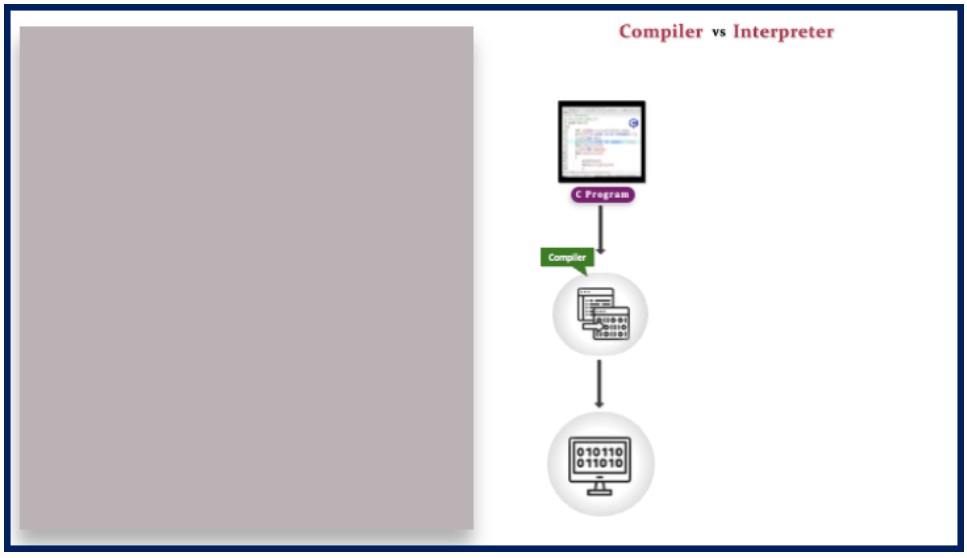
Compiler	Interpreter
• One time Translation	• 'N' time Translation
• Copy of Recipe in English	• Recipe still in Chinese
• Preparation after Translation	• Preparation while Translation
• Independent Preparation	• Preparation with Help
• If Errors, no Translation	• Partial Preparation till Error

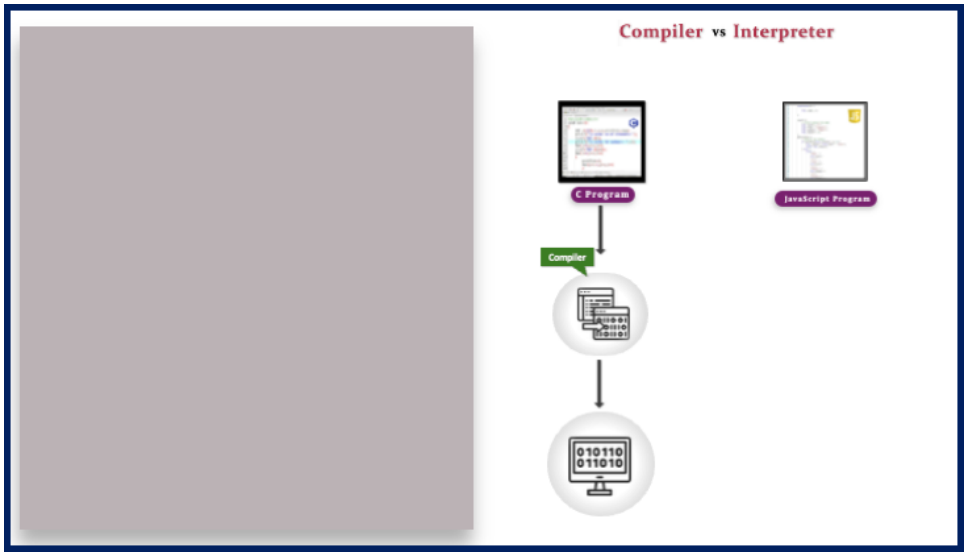
Compiler vs Interpreter

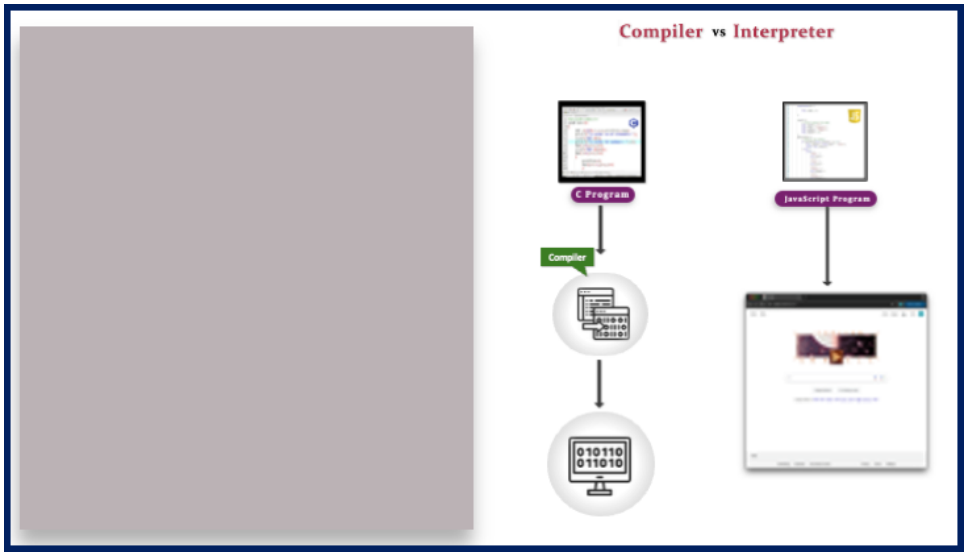

Compiler vs Interpreter
C Program

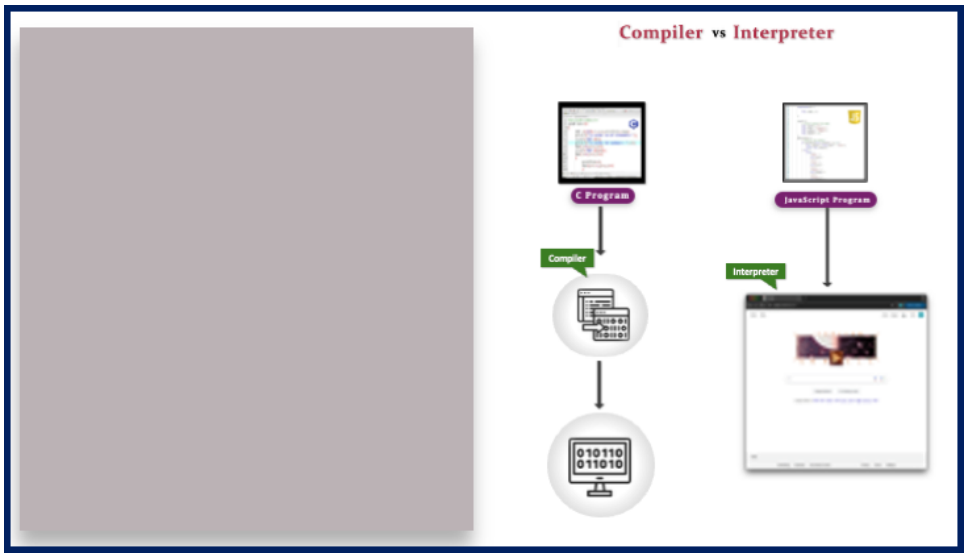

Compiler vs Interpreter
C Program
Compiler

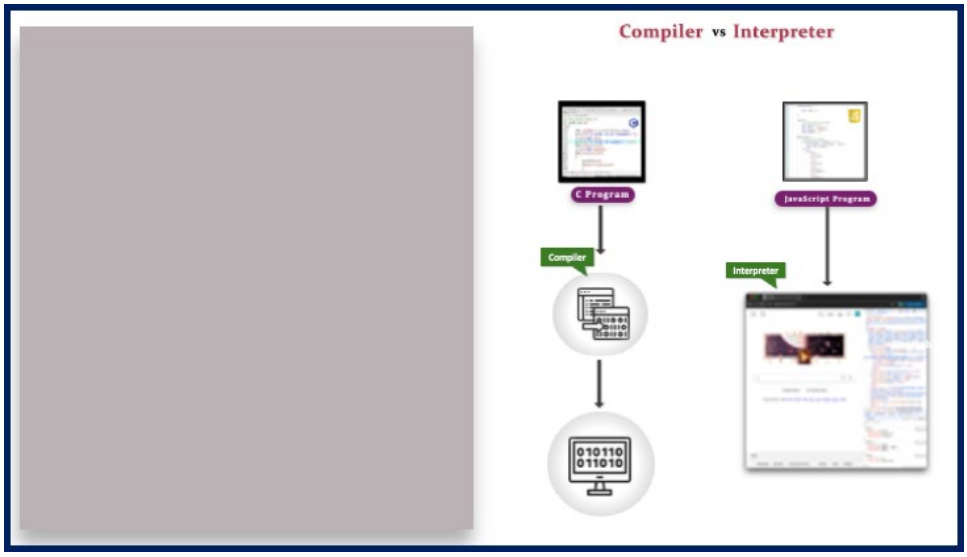

Compiler vs Interpreter
C Program
Compiler
010110
011010

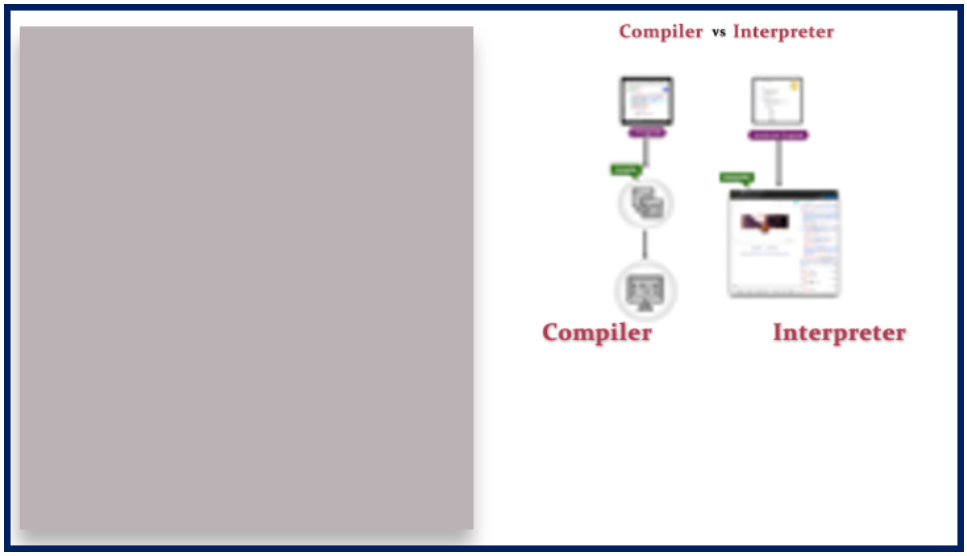

Compiler vs Interpreter
C Program
Compiler
010110
011010

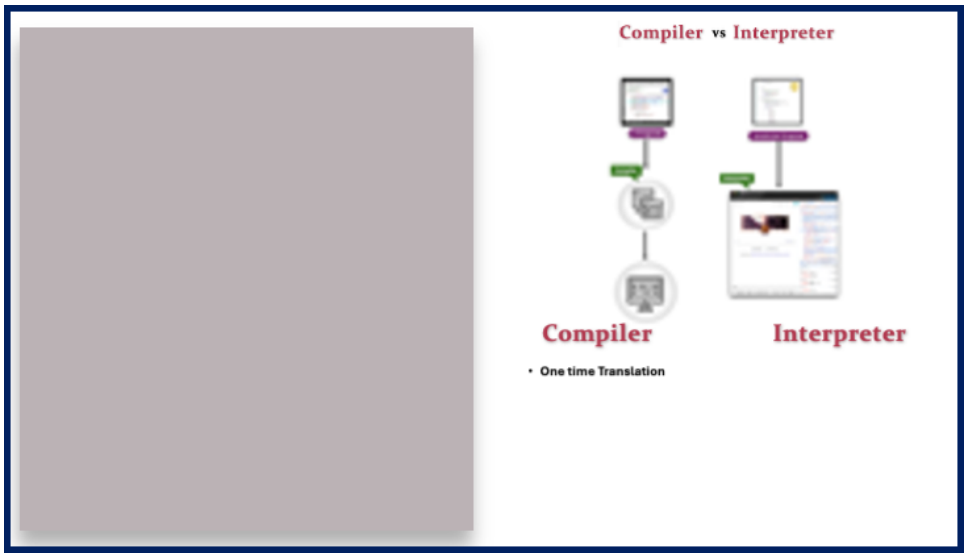

Compiler vs Interpreter
C Program
JavaScript Program
Compiler
010110
011010

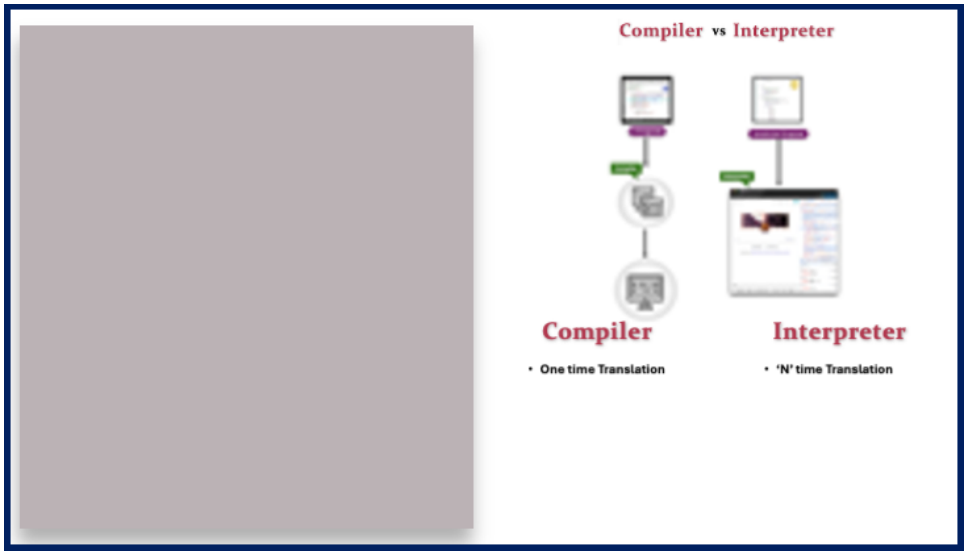

Compiler vs Interpreter
C Program
JavaScript Program
Compiler
010110
011010


Compiler vs Interpreter
C Program
Compiler
010110
011010
JavaScript Program
Interpreter


Compiler vs Interpreter
C Program
Compiler
010110
011010
JavaScript Program
Interpreter


Compiler vs Interpreter
Compiler
Interpreter


Compiler vs Interpreter
Compiler
Interpreter
• One time Translation


Compiler vs Interpreter
Compiler
Interpreter
• One time Translation
• 'N' time Translation

Compiler vs Interpreter

Compiler

- One time Translation
- File in Machine Code

Interpreter

- 'N' time Translation

Compiler vs Interpreter

Compiler

- One time Translation
- File in Machine Code

Interpreter

- 'N' time Translation
- File still in Source Code

Compiler vs Interpreter

Compiler

- One time Translation
- File in Machine Code
- Execution after Translation

Interpreter

- 'N' time Translation
- File still in Source Code

Compiler vs Interpreter

Compiler

- One time Translation
- File in Machine Code
- Execution after Translation

Interpreter

- 'N' time Translation
- File still in Source Code
- Execution while Translation

Compiler vs Interpreter

Compiler

- One time Translation
- File in Machine Code
- Execution after Translation
- Independent Execution

Interpreter

- 'N' time Translation
- File still in Source Code
- Execution while Translation

Compiler vs Interpreter

Compiler

- One time Translation
- File in Machine Code
- Execution after Translation
- Independent Execution

Interpreter

- 'N' time Translation
- File still in Source Code
- Execution while Translation
- Execution in Runtime Env.

Compiler vs Interpreter

Compiler

- One time Translation
- File in Machine Code
- Execution after Translation
- Independent Execution
- If Errors, no Compilation

Interpreter

- 'N' time Translation
- File still in Source Code
- Execution while Translation
- Execution in Runtime Env.

Compiler vs Interpreter

Compiler

- One time Translation
- File in Machine Code
- Execution after Translation
- Independent Execution
- If Errors, no Compilation

Interpreter

- 'N' time Translation
- File still in Source Code
- Execution while Translation
- Execution in Runtime Env.

- Partial Execution till Error